

Dynamic To-Do List Application

Objective

Create a **To-Do List Application**, DOM manipulation, event handling, and `localStorage`.

Instructions:

1. HTML Layout:

- Include:
 - An **input field** for new tasks.
 - A **"Add Task" button**.
 - A list area to display tasks.
- Use semantic HTML for structure.

2. CSS Styling:

- Design a clean and responsive interface.
- Tasks should:
 - Be displayed in **card-like styles**.
 - Have distinct styles for **active** and **completed** states.
- Buttons (Add, Edit, Delete, Complete) should have **hover effects** for better usability.

3. JavaScript Functionalities:

- **Add Tasks:**
 - Capture input and add the task to the list.
 - Validate that the input is not empty.
- **Edit Tasks:**
 - Allow users to modify task content via an "Edit" button.
- **Delete Tasks:**
 - Remove tasks using a "Delete" button (with confirmation).
- **Mark as Complete:**
 - Use a "Complete" button to change the task's status.
 - Move completed tasks to a separate section.
- **Persist Tasks:**
 - Save tasks in `localStorage` so they remain after refreshing the page.

4. User-Friendly Features:

- Autofocus the input field when the page loads or a task is added.
 - Prevent duplicate tasks.
 - Smooth animations for adding/deleting tasks.
-

Steps to Follow

1. **HTML:**

- Build the basic structure: Input field, "Add Task" button, and task list area.

2. **CSS:**

- Style the layout for clean visuals.
- Add hover effects for buttons.
- Separate styles for active and completed tasks.

3. **JavaScript:**

- Add tasks to the list dynamically.
- Implement "Edit", "Delete", and "Complete" button functionalities.
- Save tasks in `localStorage` and retrieve them on page load.

4. **Test:**

- Verify all features work as expected, including persistence and edge cases like empty inputs.